PTX-JIT Studio

Complete Beginner-to-Interview Project Handbook

A detailed, step-by-step explanation of the project—from what a compiler is to generated PTX, GPU execution, tests, frontend, debugging, and interview preparation.

Repository: github.com/Nadipalli-Sandeep/PTX-JIT-Studio

Frontend branch: feature/local-frontend

Frontend commit: d51bff0

Validated environment: Ubuntu 24.04, NVIDIA GeForce RTX 3050 (compute capability 8.6), driver 580.178.04

> Written for a reader with no compiler or GPU-programming background. The guide explains implemented behavior and explicitly labels limitations and future ideas.

Project-specific technical learning document

A step-by-step explanation of the project, from zero background to explaining it in an interview

Repository: https://github.com/Nadipalli-Sandeep/PTX-JIT-Studio Frontend branch: `feature/local-frontend` Frontend commit: `d51bff0` Target environment used for validation: Ubuntu 24.04, NVIDIA GeForce RTX 3050 (compute capability 8.6), CUDA driver 580.178.04.

> **Scope and honesty note.** This guide explains the implemented scalar-expression compiler and its local Streamlit frontend. The project is a focused learning prototype, not a production compiler, not a general-purpose C/C++ compiler, and not a tensor framework. The separate typed-IR experiment is not part of the committed frontend branch described here.

Part 1 — Start from absolute zero

1. What problem does this project solve?

Imagine writing this tiny mathematical formula:

```
x * 2 + 3
```

If `x = 4`, the answer is `11`. A human can calculate that easily. A computer can also calculate it, but a GPU does not simply accept an arbitrary text formula as a ready-to-run GPU program. Some software must translate the formula into a form the GPU execution toolchain understands.

PTX-JIT Studio is a small system that demonstrates that translation process. You enter a supported formula. The C++ compiler program reads the characters, understands their mathematical structure, optionally simplifies that structure, and emits NVIDIA PTX assembly. NVIDIA's assembler (`ptxas`) can assemble the generated PTX for a GPU target. The project's GPU runner then uses the CUDA Driver API to load the compiled GPU code and execute it. The program also evaluates the same formula on the CPU and compares the two answers.

The Streamlit frontend gives this process a visual interface: instead of remembering commands, a user can type an expression, choose a value for `x`, compile it, inspect the PTX, and view the CPU/GPU comparison.

What the project is—and is not

It **is**:

• A small C++17 compiler for a limited arithmetic-expression language.

• A demonstration of parsing, an abstract syntax tree (AST), simple AST optimization, and PTX text generation.

• A GPU execution and CPU-reference comparison demo using the CUDA Driver API.

• A local Streamlit interface around the existing command-line tools.

• A testable engineering project with CPU tests, randomized optimizer-equivalence checks, PTX assembly validation, and a GPU demonstration.

It **is not**:

• A compiler for arbitrary C, C++, Python, or CUDA source code.

• A general-purpose optimizing compiler comparable to LLVM or NVCC.

• A compiler for arbitrary tensor operations or complete neural networks.

• A complete production JIT runtime with sophisticated memory management, register allocation, a multi-platform backend, or industrial-grade diagnostics.

• Proof that every possible expression, GPU, or CUDA setup is supported.

2. What do the important words mean?

Source expression: The formula the user types, such as `x * 2 + 3`.

Compiler: A program that translates a program written in one language into another representation.

Frontend: The part that accepts user input and presents results. In this project, the Streamlit app is the visible frontend; the expression parser is also often called the language frontend in compiler terminology. These are two different uses of

the word.

Parser: Code that recognizes the grammar of the input and determines how the expression is grouped.

AST (Abstract Syntax Tree): A tree-shaped representation of the expression's meaning. It captures operations and operands, not merely the original characters.

Optimization: A transformation that tries to make the representation simpler or cheaper while preserving the result.

Code generation: Turning the internal representation into instructions for another system. Here, that output is PTX text.

PTX: NVIDIA's Parallel Thread Execution virtual instruction set. It is a GPU-oriented assembly-like representation understood by NVIDIA's GPU toolchain. It is not the same as machine code for a particular GPU.

`ptxas`**:** NVIDIA's assembler that translates PTX into GPU-targeted machine code, commonly stored as a cubin.

CUDA Driver API: NVIDIA's lower-level API for initializing the CUDA driver, loading GPU code, managing device memory, launching kernels, and retrieving results.

Kernel: A function that runs on the GPU. This project emits a kernel named `ptxjit_kernel`.

CPU reference: A CPU calculation used as a comparison point. It helps detect mistakes in the GPU result.

Test: A repeatable check that the program behaves as expected for chosen inputs.

JIT (just-in-time compilation): In a broad sense, compilation performed during program execution or on demand. This project demonstrates an on-demand expression-to-PTX workflow and GPU execution. Avoid claiming that it implements every feature of a mature JIT runtime.

3. The entire journey in one picture

```
User enters: x * 2 + 3
          |
          v
Streamlit web interface (app.py)
          |
          v
C++ command-line compiler (build/ptxjit)
          |
          v
Characters -> tokens / parser decisions -> AST
          |
          v
AST optimization (constant folding / identities)
          |
          v
PTX code generator
          |
          v
Generated .ptx text
          |
          v
NVIDIA ptxas assembles PTX for sm_86
          |
          v
GPU machine-code file (cubin)
          |
          v
CUDA Driver API runner loads and launches kernel
          |
          v
GPU result --------------------- CPU reference result
          |                                |
          +----------- compare ------------+
                         |
                         v
             Frontend shows output and error
```

Each box has a separate responsibility. This separation makes the project easier to debug: if parsing fails, inspect the parser; if PTX is invalid, inspect generated PTX and assembler errors; if the GPU result is wrong, compare it with the CPU result and inspect the kernel/runtime path.

Part 2 — Understand the language the compiler accepts

4. The supported expression language

The implemented compiler handles scalar arithmetic expressions built from:

• The variable `x`.

• Floating-point numeric constants.

• Addition: `+`.

• Subtraction: `-`.

• Multiplication: `*`.

• Division: `/`.

• Unary plus and unary minus.

• Parentheses to control grouping.

Examples of the intended input language:

```
x
4
x + 5
x * 2 + 3
(x + 5) * 2
-x
--x
x / 2 - 1
(2 + 3) * (x - 4)
```

Examples outside the documented scope include function calls such as `sin(x)`, loops, conditionals, arrays, multiple variables such as `y`, assignment statements, and arbitrary C++ syntax. Do not assume these work unless you have specifically extended and tested the parser.

Why the supported language is deliberately small

A small language makes it possible to study the core compiler pipeline without first implementing a large programming language. Even this simple language requires important compiler concepts: precedence, associativity, tree structure, safe transformations, instruction selection, code generation, and runtime execution.

5. Why parentheses and operator precedence matter

Consider:

```
x + 2 * 3
```

Normal arithmetic rules say multiplication happens before addition. The intended grouping is:

```
x + (2 * 3)
```

It is **not**:

```
(x + 2) * 3
```

With `x = 4`, the first expression gives `10`, while the second gives `18`. A parser must therefore understand precedence instead of just reading characters left to right.

Parentheses can explicitly change grouping:

```
(x + 2) * 3
```

Now the addition is grouped first because the parentheses say so.

Subtraction and division also demonstrate why grouping matters:

```
x - 2 - 1
```

For a conventional left-associative grammar this means `(x - 2) - 1`, not `x - (2 - 1)`. Similarly, `x / 2 / 3` normally groups as `(x / 2) / 3`.

6. Tokens and parsing, explained simply

A **token** is a meaningful piece of input. For example, the text:

```
x * 2 + 3
```

contains pieces resembling:

```
VARIABLE(x), STAR(*), NUMBER(2), PLUS(+), NUMBER(3)
```

The exact implementation can read characters directly or use token objects; the conceptual point is that the parser must recognize variable names, numbers, and operators.

A recursive-descent parser uses a set of small functions, each responsible for a level of grammar. A typical structure is:

• Parse an expression with addition and subtraction.

• Parse a term with multiplication and division.

• Parse a unary expression with leading signs.

• Parse a primary expression such as a number, variable, or parenthesized expression.

The exact function names are implementation details. The important design is that higher-level grammar functions call lower-level functions to enforce precedence.

A walk-through

Input:

```
x * 2 + 3
```

The parser begins reading an expression.

It recognizes `x` as a primary value.

It sees `*`, so it parses the right-hand value `2` and creates a multiplication node.

It then sees `+`, which has lower precedence than multiplication.

It parses `3` and creates an addition node whose left child is the multiplication node.

The resulting tree represents `(x * 2) + 3`.

If the parser encounters a missing closing parenthesis, an unsupported identifier, or an unexpected symbol, it should report an error rather than quietly generate a different expression. Error quality depends on the current implementation; do not claim a full industrial-grade diagnostic system.

Part 3 — The AST and why it matters

7. What is an Abstract Syntax Tree?

An AST is a tree that describes what an expression means.

For:

```
x * 2 + 3
```

the conceptual tree is:

```
        (+)
       /   \
     (*)    3
    /   \
   x     2
```

The top node is addition. Its left child is multiplication. Its right child is the constant `3`. Multiplication has `x` and `2` as children.

The tree preserves the important grouping. It does not need to preserve every space or the exact spelling of the original input.

For:

```
(x + 2) * 3
```

the tree is different:

```
        (*)
       /   \
     (+)    3
    /   \
   x     2
```

That difference is why a tree is more useful than a flat string when performing optimization and code generation.

8. What the AST makes easier

Without an AST, a code generator might repeatedly inspect raw characters and attempt to work out the grouping again. That mixes parsing and code generation and makes mistakes more likely.

With an AST:

• The parser decides how the expression is grouped.

• The optimizer examines nodes and their children.

• The PTX generator recursively translates those nodes into instructions.

• The CPU evaluator can independently calculate the tree's value.

These stages have clear jobs and can be tested separately.

Part 4 — AST optimization

9. What is optimization?

Optimization means changing the program representation without changing the answer for inputs where the transformation is valid.

A simple example is:

```
2 + 3
```

Since both operands are constants, the compiler can calculate `5` before running the GPU kernel. This is **constant folding**.

Another example:

```
x + 0
```

For ordinary finite arithmetic values, adding zero leaves the value unchanged. An optimizer may simplify the tree to just `x`. This is an example of an algebraic identity or identity simplification.

Optimization is not automatically correct just because a transformation looks mathematically familiar. Floating-point arithmetic has rounding, infinities, NaNs, signed zeros, and division-by-zero behavior. A transformation that is safe under simple real-number arithmetic may change observable floating-point behavior in some edge cases. A small learning compiler should use conservative rules and document its numeric assumptions.

10. Constant folding

Suppose the input is:

```
x * (2 + 3)
```

The original tree contains a constant addition inside the multiplication. The optimizer can replace `(2 + 3)` with `5`, yielding:

```
x * 5
```

The CPU and GPU then need not perform that constant addition at runtime.

Another example:

```
(2 * 3) + (4 * 5)
```

can be folded into:

```
6 + 20
```

and then into:

```
26
```

The expression becomes a constant. Depending on the generated kernel and compiler design, the runtime may still execute a very small kernel to produce the result; the key fact is that the arithmetic has been resolved during compilation.

11. Identity simplification

Common identities include:

• `a + 0` becomes `a`.

• `a - 0` becomes `a`.

• `a * 1` becomes `a`.

• `a / 1` becomes `a`.

• `a * 0` may become `0` under appropriate numeric assumptions.

The final rule needs more care than it first appears to. For example, if `a` could be NaN or infinity, replacing `a * 0` with `0` can change behavior. A responsible compiler engineer asks not just “is this identity true in algebra?” but “is it valid under the language's actual numeric semantics?”

The project's optimizer implements constant folding and algebraic simplification. For interview discussion, explain the goal and examples, then acknowledge floating-point edge cases as an area requiring conservative design and additional tests.

12. How to test an optimizer

A dangerous optimizer bug is one that returns a plausible but incorrect result. One useful strategy is **differential equivalence testing**:

Generate an expression.

Evaluate the original expression.

Optimize the expression.

Evaluate the optimized expression.

Compare the results for several input values.

The project's randomized optimizer-equivalence tests exercise 300 generated expressions at 7 input values each, for 2,100 checks. This gives broader coverage than a handful of manually chosen expressions, but it is not a mathematical proof that every possible expression is correct.

A good test suite combines:

• Simple examples with known answers.

• Precedence and parenthesis cases.

• Unary signs.

• Constant-only expressions.

• Identity simplifications.

• Randomized equivalence checks.

• Invalid-input tests where supported.

• GPU-versus-CPU checks.

Part 5 — Generate PTX

13. Why generate PTX instead of directly generating GPU machine code?

GPU machine code is specific to GPU architectures and is not a convenient format to hand-author for a small project. PTX is a virtual instruction set used in NVIDIA's toolchain. The project can emit readable PTX text, inspect it, and pass it to NVIDIA's assembler.

This division is important:

• **PTX generation** is the project's code-generation responsibility.

• **Assembly for a target architecture** is performed by NVIDIA's `ptxas`.

• **Loading and executing the resulting GPU code** is performed through the CUDA Driver API.

PTX is not simply a text form of the exact final machine instructions. The assembler and toolchain translate it for a particular target.

14. What does a PTX kernel look like?

The generated PTX uses a kernel named `ptxjit_kernel`. The exact register names and instruction sequence depend on the expression and generator. The following is a **conceptual illustration**, not a promise of the exact output text for every build:

```
.version 7.0
.target sm_50
.address_size 64

.visible .entry ptxjit_kernel(
    .param .u64 output_ptr,
    .param .f32 x_value
)
{
    // Conceptual steps:
    // 1. Read x_value.
    // 2. Multiply x by 2.
    // 3. Add 3.
    // 4. Store the answer through output_ptr.
    // 5. Finish the kernel.
    ret;
}
```

The actual implementation's parameter layout and instruction details must be read from the generated `.ptx` file. The snippet above explains the shape of a kernel but is not intended to assemble as-is.

Important PTX concepts

• `.version` declares the PTX language version.

• `.target` identifies the virtual target capability used by the PTX module.

• `.address_size 64` declares 64-bit addressing.

• `.entry` defines a kernel entry point.

• `.param` declares a kernel parameter.

• Registers hold temporary values.

• Instructions perform arithmetic, memory operations, and control flow.

• `ret` returns from the kernel.

The project emits `.version 7.0`, `.target sm_50`, and 64-bit addressing in its PTX output. The validation script assembles representative generated PTX with `ptxas -arch=sm_86`. Do not confuse the PTX module's declared target with the architecture argument passed to `ptxas`; they are different parts of the toolchain configuration.

15. What does `ptxas` do?

`ptxas` is NVIDIA's PTX assembler. It reads PTX text, checks it, and creates GPU-targeted code in a cubin file.

A simplified sequence is:

```
generated.ptx
    |
    | ptxas -arch=sm_86
    v
generated.cubin
```

If the PTX has an invalid instruction or incompatible declaration, assembly can fail and provide a diagnostic. This is why assembling generated PTX is a separate validation step: successfully printing text does not prove that the text is valid PTX.

The project's `scripts/validate_ptx.sh` checks six representative expressions by invoking `ptxas` for `sm_86`. Passing these checks means those sample outputs assembled successfully in the tested environment. It does not prove that every possible expression or every NVIDIA architecture is supported.

Part 6 — Execute on the GPU

16. What is a GPU kernel?

A GPU kernel is a function launched on the GPU. GPUs are designed to execute many operations in parallel. This project uses a deliberately simple scalar expression and a small kernel so the compilation and runtime path can be studied without the complexity of a large parallel algorithm.

The project demonstrates GPU execution; it is not intended to prove that a tiny one-value expression runs faster on a GPU than on a CPU. For a tiny workload, setup and launch overhead can dominate the arithmetic.

17. CUDA Driver API, explained step by step

The CUDA Driver API is a lower-level interface for controlling GPU execution. Conceptually, a runner must perform steps like these:

Initialize the CUDA driver.

Select or create a CUDA context for the device.

Load the compiled GPU module or cubin.

Find the `ptxjit_kernel` entry point.

Allocate device memory for input/output data as required.

Prepare kernel arguments.

Launch the kernel.

Synchronize when necessary so execution has completed.

Copy the result back to host memory.

Release resources and compare the result with the CPU reference.

This is a conceptual lifecycle. The exact sequence, error handling, resource lifetime, and parameter layout should be verified in the repository's GPU runner source before describing each individual API call in an interview.

CPU memory versus GPU memory

The CPU and GPU do not automatically share every ordinary variable. A CPU variable named `x` is not automatically a device-memory pointer. The runner must prepare the values in the form expected by the kernel and retrieve the output when GPU work has completed.

This host/device boundary is one reason GPU programming is more involved than calling a normal C++ function.

18. CPU reference and correctness check

The project evaluates the expression on the CPU as a reference and compares it with the GPU output.

Example:

```
Expression: x * 2 + 3
Input x:    4
CPU result: 11
GPU result: 11
Absolute error: 0
```

A common comparison measure is absolute error:

\[\text{absolute error} = |\text{GPU result} - \text{CPU result}| \]

A zero error means the displayed values match exactly for that test. Floating-point computations can sometimes differ slightly because of rounding or instruction choices, so a robust comparison may use a tolerance:

\[

a-b	\leq \text{absolute tolerance}

+ \text{relative tolerance}\cdot |b| \]

The correct tolerance depends on the numeric type and intended semantics. Do not claim a particular tolerance policy unless the implementation actually uses it.

19. Why CPU/GPU comparison is useful

The CPU result gives a second implementation against which to check the GPU path. If they differ, possible causes include:

• Incorrect parsing or operator precedence.

• A faulty optimization.

• Incorrect PTX instruction selection or register handling.

• A kernel parameter mismatch.

• A memory-copy or synchronization problem.

• Different floating-point behavior.

• A bug in the CPU reference itself.

A comparison does not automatically tell you which part is wrong. It narrows the investigation to the expression and execution path, after which the generated AST, PTX, assembler diagnostics, and runtime handling can be inspected.

Part 7 — The Streamlit frontend

20. What does the frontend add?

The frontend is implemented in `app.py` using Streamlit. It is a local web interface around the existing command-line compiler and GPU runner; it does not replace the compiler.

The app provides a developer-dashboard-style interface with:

• Expression input and a value for `x`.

• A compile/run workflow.

• PTX assembly inspection.

• Compiler and environment details.

• GPU execution output and CPU/GPU comparison.

• An expression guide.

• Status and metric panels.

The app invokes real local tools rather than inventing a PTX result in the browser. It runs the compiler executable, uses `ptxas` for assembly validation, and calls the GPU runner when the environment supports it. It uses temporary files for generated artifacts so repeated runs do not need fixed filenames in the repository.

21. What happens when the user clicks the run button?

At a conceptual level:

Streamlit reads the expression and numeric input.

Python launches the compiled C++ compiler process.

The compiler produces PTX or reports an error.

The frontend stores/reads the generated PTX in a temporary working area.

`ptxas` is invoked for the configured target architecture.

If assembly succeeds and GPU execution is available, the GPU runner is invoked.

The app reads process output and exit codes.

It extracts the CPU result, GPU result, and comparison details where available.

The page displays PTX, logs, and result metrics.

This separation keeps Python responsible for the interface and orchestration while C++ owns expression compilation and the GPU runner owns the runtime execution path.

22. Why Streamlit?

Streamlit makes it possible to build a local interactive application in Python without creating a separate JavaScript frontend and web backend. That is useful for a compiler demo because the engineering effort stays focused on the compiler and GPU workflow.

The trade-off is that Streamlit is designed for data apps and prototypes. It is not, by itself, a secure multi-user production service or a substitute for robust sandboxing of arbitrary untrusted compiler inputs. This project should be run as a local development tool.

23. Run the frontend

From the project root:

```
cd ~/projects/PTX-JIT-Studio
source .venv/bin/activate
streamlit run app.py --server.address 0.0.0.0
```

Open the URL printed by Streamlit, normally:

```
http://localhost:8501
```

Binding to `0.0.0.0` lets Streamlit listen on network interfaces, which can be useful when opening it from another device on the same network. Only do this on a trusted network and do not expose the development server to the public internet without appropriate security controls.

If the virtual environment has not yet been created:

```
cd ~/projects/PTX-JIT-Studio
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install -r requirements-frontend.txt
```

The C++ compiler and CUDA tools are separate prerequisites. Installing Python dependencies alone does not install `ptxas`, a CUDA driver, or build the C++ binaries.

Part 8 — Project layout and responsibility of each file

The repository contains C++ source, public headers, tests, scripts, CMake build configuration, documentation, and the Streamlit app. The exact file listing may evolve; use `find` or `git ls-files` to see the current checkout.

Important paths include:

Path	Purpose
`app.py`	Streamlit user interface and orchestration of local tools
`requirements-frontend.txt`	Python dependencies for the frontend
`CMakeLists.txt`	Describes how CMake configures and builds the C++ project
`include/ptxjit/`	Public C++ declarations and interfaces
`src/`	C++ implementation for compiler and runtime components
`tests/compiler_tests.cpp`	Compiler-focused tests, including randomized optimizer equivalence checks
`scripts/demo.sh`	Reproducible command-line demonstration
`scripts/validate_ptx.sh`	PTX assembly validation for representative expressions
`.github/workflows/ci.yml`	CPU-oriented automated CI workflow
`README.md`	Project overview, build/run instructions, and frontend information
`build/`	Generated build output; should generally not be committed
`.venv/`	Local Python environment; should not be committed

To inspect the real source layout:

```
git ls-files
```

To find all files under a directory:

```
find include src tests scripts -maxdepth 3 -type f
```

Why headers and source files are separated

A header usually declares an interface: the types and function signatures other code can use. A `.cpp` file usually contains implementation details. This separation lets different components depend on stable declarations instead of needing every implementation detail.

Why CMake is used

CMake describes how to configure and build the C++ project. It detects compilers and generates build files. It does not itself execute the compiler program or prove correctness; build and test commands are separate steps.

Part 9 — Build and test from a clean checkout

24. Required tools

The exact installation method varies by operating system. On the validated Linux setup, the project uses:

• A C++17-capable compiler such as GCC.

• CMake.

• Python 3 and `venv` for the frontend.

• NVIDIA GPU driver and CUDA tools, including `ptxas`.

• An NVIDIA GPU supported by the installed driver/toolchain for actual GPU execution.

Check tools:

```
g++ --version
cmake --version
python3 --version
ptxas --version
nvidia-smi
```

A machine without an NVIDIA GPU can still build and run CPU-side tests if the project configuration supports that path, but it cannot reproduce the actual GPU execution demonstration.

25. Configure and build

```
cd ~/projects/PTX-JIT-Studio
cmake -S . -B build
cmake --build build -j2
```

What these commands mean:

• `cmake -S . -B build`: use the current directory as the source and `build` as the generated build directory.

• `cmake --build build`: ask CMake to invoke the configured build tool.

• `-j2`: allow up to two build jobs in parallel.

If a build fails, read the first meaningful compiler error rather than only the final “build failed” line. Later errors may be consequences of the first one.

26. Run automated tests

```
ctest --test-dir build --output-on-failure
```

`ctest` runs tests registered by the CMake project. `--output-on-failure` shows details for failing tests. On the validated frontend branch, the recorded run passed 1/1 registered CTest test. The C++ test executable also contains the randomized optimizer-equivalence checks described earlier.

A successful test run is evidence for the tested cases, not proof that no bugs remain.

27. Validate generated PTX

```
./scripts/validate_ptx.sh
```

The validation script assembles six representative generated PTX cases using `ptxas -arch=sm_86`. The recorded validation passed all six.

If it fails:

Read the first `ptxas` diagnostic.

Keep the generated PTX for the failing expression.

Check the emitted instruction, operand types, registers, parameter declarations, and target configuration.

Re-run the smallest failing case after fixing it.

28. Run the GPU demo

```
./scripts/demo.sh "x * 2 + 3" 4
```

The recorded expected result is GPU `11`, CPU `11`, absolute error `0`. This is a smoke test for the end-to-end path. If the GPU runner fails, first verify `nvidia-smi`, the presence of `ptxas`, the build output, and any CUDA driver/runtime diagnostics.

29. Run the frontend

```
source .venv/bin/activate
streamlit run app.py --server.address 0.0.0.0
```

In the browser:

Enter `x * 2 + 3`.

Set `x` to `4`.

Run the workflow.

Inspect the generated PTX tab.

Check compiler/assembly details.

Check GPU output and CPU reference.

Confirm the displayed comparison is `11` versus `11` and absolute error `0`.

Try a second expression such as `(x + 5) * 2` with `x = 4`, which should evaluate to `18`.

Try an intentionally malformed expression to see how errors are displayed.

Confirm the app returns to a usable state after an error.

The browser workflow must be tested separately; Python syntax checking does not prove that every UI interaction works.

30. Useful verification commands

```
git status --short --branch
git diff --check
python -m py_compile app.py
cmake --build build -j2
ctest --test-dir build --output-on-failure
./scripts/validate_ptx.sh
./scripts/demo.sh "x * 2 + 3" 4
```

`git diff --check` detects whitespace errors in changes. `python -m py_compile app.py` checks Python syntax but does not launch the app. The CMake, CTest, PTX, and GPU demo commands validate different layers, so no single command replaces all the others.

Part 10 — Performance and benchmarks

31. What should a benchmark measure?

The project includes compiler and GPU microbenchmark scripts. Benchmark numbers only mean something when the measurement is defined. Possible measures include:

• Compilation time for an expression.

• PTX assembly time.

• Kernel launch and execution time.

• End-to-end time including compilation, assembly, loading, and execution.

• CPU evaluation time.

These are different measurements. A GPU kernel might execute quickly once loaded, while the total workflow takes longer because compilation, assembly, context setup, memory transfers, and launch overhead are included.

Do not invent performance numbers. Run the repository's benchmark scripts and report the exact command, machine, GPU, driver/toolchain version, input expressions, number of repetitions, and whether setup overhead is included.

32. Why this project is not a GPU speedup demonstration

The expression `x * 2 + 3` contains very little work. A GPU has startup and launch overhead, so it is not reasonable to expect this single calculation to beat a CPU calculation in total latency. The educational value is that the project demonstrates the compilation and execution path.

A fair interview answer is: “The current workload is intentionally tiny to make compiler mechanics visible. It demonstrates correct end-to-end execution and inspectable PTX; it is not designed to show a performance win for one scalar operation.”

Part 11 — CI, Git, and project delivery

33. What does CI do?

Continuous Integration (CI) runs repeatable checks automatically when code is pushed or a pull request is opened. This repository has a CPU-oriented GitHub Actions workflow. A CPU-only CI workflow is useful because not every hosted runner has an NVIDIA GPU or the CUDA toolchain required for GPU execution.

Therefore:

• CPU tests can be run in ordinary CI.

• PTX assembly validation needs `ptxas`.

• Real GPU execution needs an appropriate GPU/driver environment.

• A green CPU CI check should not be described as proof that GPU execution ran in CI.

34. Branches and commits

A branch is a separate line of project history. The frontend work is on:

```
feature/local-frontend
```

The frontend commit is:

```
d51bff0 Add local Streamlit frontend for PTX-JIT Studio
```

The branch was pushed to GitHub. The `main` branch was deliberately not changed by this frontend work. The GitHub page for the branch is:

https://github.com/Nadipalli-Sandeep/PTX-JIT-Studio/tree/feature/local-frontend

A pull request can be opened for review, but pushing a branch does not automatically mean it has been merged into `main`.

Useful Git commands:

```
git branch --show-current
git status --short --branch
git log --oneline -5
git diff --check
git remote -v
```

Before committing:

Inspect the changed files.

Run the relevant tests.

Confirm generated build files, virtual environments, secrets, and temporary artifacts are not staged.

Commit with a clear message.

Push the intended branch.

Verify the branch and files on GitHub.

Never put a personal access token, password, or private credential in source code, a README, a commit message, or this handbook.

Part 12 — Common errors and how to reason about them

35. `ptxas: command not found`

Likely cause: CUDA assembler is not installed or is not on `PATH`.

Check:

```
command -v ptxas
ptxas --version
```

Install/configure the appropriate NVIDIA CUDA toolkit for the system, then open a new shell or update the environment. A GPU driver alone does not guarantee that `ptxas` is installed.

36. `nvidia-smi` fails

Likely causes include a driver problem, a system without an NVIDIA GPU, or an unavailable GPU device. Check the driver installation and system logs. Do not diagnose this as a compiler bug until the hardware/driver environment is checked.

37. CMake cannot find a dependency

Read the first configuration error. Verify that the required development package is installed, then re-run:

```
cmake -S . -B build
cmake --build build -j2
```

If the build directory contains stale configuration after a major toolchain change, create a fresh build directory rather than deleting source files.

38. PTX generation succeeds but `ptxas` fails

Text generation and assembly are separate. Save the exact generated PTX and inspect the first assembler diagnostic. Common categories include malformed directives, unsupported instructions, invalid operand types, incorrect register usage, and target incompatibility.

39. GPU output differs from CPU output

Reduce the expression to the smallest failing example. Then inspect:

The input value and parsed grouping.

The optimized expression.

Generated PTX.

Kernel parameter layout and data types.

Device-memory allocation and transfers.

Kernel launch and synchronization.

CPU evaluator behavior and floating-point edge cases.

Avoid “fixing” a discrepancy by simply loosening the comparison tolerance without understanding the cause.

40. Frontend cannot find a binary

The app expects the local C++ build output. Run:

```
cmake -S . -B build
cmake --build build -j2
```

Then launch Streamlit from the project root. Relative paths can behave differently if the app is launched from another directory; use the intended project root.

41. Port 8501 is already in use

Another Streamlit process may already be running. Stop the old process or choose another port:

```
streamlit run app.py --server.port 8502
```

Then open `http://localhost:8502`.

42. Frontend works but GPU execution is unavailable

The interface can start even when the complete GPU toolchain is not available, but the GPU execution stage requires the appropriate local binaries, `ptxas`, driver, and supported GPU. Distinguish “the web page loaded” from “the complete GPU workflow succeeded.”

Part 13 — Limitations and sensible future work

43. Current limitations

The current implementation intentionally focuses on scalar expressions. Important limitations include:

• One variable, `x`.

• A limited arithmetic grammar.

• No general function calls, loops, branches, arrays, or user-defined functions.

• No full C/C++ frontend.

• A small set of AST optimizations rather than a broad optimizing compiler pipeline.

• No claim of a complete SSA-based IR, global register allocator, or general instruction scheduler.

• Target/toolchain assumptions need to be managed carefully.

• GPU execution and frontend behavior depend on local system configuration.

• Tiny expressions are not a meaningful GPU performance workload.

• Randomized tests increase confidence but cannot prove correctness for all inputs.

44. Potential next steps (not claims about current implementation)

These are possible extensions, not features to claim as already completed:

Define explicit language semantics for floating-point edge cases.

Add more parser and invalid-input tests.

Add richer diagnostics with expression positions.

Add more data types and a carefully specified typed intermediate representation.

Add explicit target configuration and architecture compatibility checks.

Expand the benchmark methodology and publish reproducible raw measurements.

Add CI jobs for PTX assembly where the toolchain is available.

Add GPU integration tests on a dedicated GPU runner.

Add frontend tests for success, compile failure, missing dependencies, and GPU runtime errors.

Add more complex workloads only after the language and correctness model are well defined.

A future feature should have a clear specification, implementation, tests, documentation, and measured results before it is described as completed.

Part 14 — How to explain the project in an interview

45. A 30-second explanation

“I built PTX-JIT Studio, a small C++17 compiler that accepts scalar arithmetic expressions, parses them into an abstract syntax tree, performs constant folding and algebraic simplification, and generates NVIDIA PTX. I validate generated PTX with `ptxas`, execute the assembled kernel through the CUDA Driver API, and compare its result with a CPU reference. I also built a local Streamlit interface to enter expressions, inspect PTX, and view the CPU/GPU comparison. The focus is understanding the compiler-to-GPU execution path, not claiming a production compiler or a speedup for trivial scalar arithmetic.”

46. A two-minute explanation

“The problem I wanted to understand was what happens between a high-level arithmetic expression and actual GPU execution. I limited the language to one variable, floating-point constants, arithmetic operators, unary signs, and parentheses so that I could implement the compiler pipeline end to end.

The C++17 compiler parses the expression using recursive-descent logic and builds an AST. The AST captures operator precedence, so `x * 2 + 3` is represented as addition whose left side is multiplication. An optimization pass folds constant expressions and applies selected algebraic identities. The code generator then emits PTX, NVIDIA's assembly-like virtual GPU instruction set.

I do not generate final GPU machine code myself. I invoke `ptxas` to assemble the PTX for the target architecture, then a GPU runner uses the CUDA Driver API to load and launch the kernel. I calculate a CPU reference result and compare it

with the GPU result to help validate correctness.

I tested the C++ build with CTest, used randomized optimizer-equivalence checks across 300 expressions and seven values each, validated six representative PTX outputs with `ptxas`, and demonstrated `x * 2 + 3` at `x = 4`, where both CPU and GPU return 11. I then added a Streamlit frontend that orchestrates the local tools and displays PTX and execution details. The main limitations are the intentionally small language, floating-point edge cases, and the fact that a tiny scalar workload is for learning rather than GPU speedup."

47. Questions you should be ready to answer

Q: Why did you choose a recursive-descent parser? It is a direct way to encode a small grammar in readable functions. Separate grammar levels make precedence and grouping easier to reason about.

Q: Why use an AST instead of generating PTX while reading characters? The AST separates syntax understanding from optimization and code generation. It also makes it easier to evaluate and test expressions.

Q: What is constant folding? Evaluating constant-only subexpressions during compilation, such as replacing `2 + 3` with `5`.

Q: What is algebraic simplification? Replacing an expression with an equivalent simpler form, such as `x + 0` with `x`, under the language's numeric assumptions.

Q: Why can floating-point optimization be dangerous? Floating-point has rounding, NaN, infinity, and signed-zero behavior. Real-number identities do not always preserve all floating-point behavior.

Q: What is PTX? NVIDIA's virtual GPU instruction-set representation. It is assembled for a GPU target by `ptxas`; it is not identical to final GPU machine code.

Q: Why use `ptxas`? It validates and assembles the emitted PTX into target-specific GPU code.

Q: What does the CUDA Driver API do in this project? It provides the lower-level operations needed to load the GPU module, launch the kernel, manage device-side data, and retrieve results. Confirm the exact calls in the runner source before naming them individually.

Q: Why compare against a CPU result? It provides an independent reference to catch errors across parsing, optimization, code generation, and GPU execution.

Q: Why not just use CUDA C++ or NVCC? The project is specifically a learning exercise in generating and inspecting PTX from a small custom language. A mature toolchain is appropriate for production software, but would hide the code-generation steps this project aims to demonstrate.

Q: Does the project make scalar arithmetic faster on the GPU? That is not the goal. For tiny expressions, GPU launch and setup overhead can dominate. The project demonstrates the execution pipeline and correctness checks.

Q: What is the role of Streamlit? It provides a local UI that orchestrates the compiler and runtime tools and displays their outputs. It is not the compiler itself.

Q: What does the randomized test demonstrate? It checks that the optimizer and original expression evaluator agree for 300 generated expressions at seven values each. It improves confidence but does not prove correctness for every possible expression.

Q: What is the difference between PTX generation and PTX assembly? The project's generator emits PTX text; NVIDIA's `ptxas` translates that text into target-specific GPU code.

Q: What is the difference between a successful CPU test and a GPU integration test? A CPU test can run without an NVIDIA GPU. A GPU integration test also depends on the driver, device, CUDA tools, and the GPU runner.

Q: What would you improve next? I would formalize numeric semantics, expand edge-case tests, improve diagnostics, make target handling more explicit, and add repeatable GPU integration tests. I would implement one extension at a time and verify it before claiming it as complete.

48. Questions where you should be precise, not overclaim

If asked whether this is a production JIT compiler, answer no: it is a focused compiler prototype.

If asked whether it supports arbitrary CUDA or C++, answer no: it supports a small scalar-expression grammar.

If asked whether the typed IR experiment is part of the delivered frontend branch, answer no: it was a separate experimental branch/stash and is not part of the committed frontend branch.

If asked whether CI executes GPU tests, explain that the configured CI is CPU-oriented; the GPU workflow was validated locally on the NVIDIA setup.

If asked whether it achieves a speedup, explain that the demo is too small to establish one and that the objective is correctness and transparency of the compilation pipeline.

If asked how to reproduce the result, give the exact build, test, PTX validation, and demo commands in Part 9.

Part 15 — Final reproducibility checklist

Use this checklist before an interview or demo.

• [ ] Repository is on the intended branch.

• [ ] `git status --short --branch` shows no unexpected modifications.

• [ ] CMake configuration and build succeed.

• [ ] CTest passes.

• [ ] PTX validation passes all six cases.

• [ ] GPU demo returns matching CPU/GPU values for a known example.

• [ ] Streamlit launches from the project root.

• [ ] Browser workflow succeeds for `x * 2 + 3`, `x = 4`.

• [ ] Generated PTX is visible and can be explained.

• [ ] You can explain the AST and why precedence matters.

• [ ] You can explain constant folding and floating-point caveats.

• [ ] You can distinguish PTX generation, `ptxas` assembly, and CUDA Driver API execution.

• [ ] You know the project's current limitations.

• [ ] Resume claims match code that is actually committed and tested.

• [ ] GitHub branch and documentation links work.

• [ ] No credentials or secrets are in the repository.

Closing perspective

The most valuable part of PTX-JIT Studio is the complete learning path: a small input language is parsed, represented as a tree, simplified, translated to PTX, assembled for a GPU target, executed through the CUDA Driver API, and checked against a CPU reference. The frontend makes that path observable. Being able to explain every stage and its limitations clearly is more valuable in an interview than adding many untested features.